

EN LOS RENDER HAY QUE PONER UN RETURN EN EL PRINCIPIO

Las funciones que hay que utilizar suelen venir importadas, sino visitar los endPoints

BOTONES

Si un botón no hace nada al darle, añadirlo en el RestaurantStack.

Si estás en el menú principal (RestaurantScreen o ProfileScreen), en el navigation.navigate de onPress no debes poner route.params.id, sino que tienes que poner loggedInUser.id

EndPoints

Para ver la uri que hay que utilizar, visitar los routes del backend

Lo primero es realizar el useEffect para ver si está logueado. Más tarde, todos los fetchs que se hagan se van introduciendo aquí

```
useEffect(() => {  
  if (loggedInUser) {  
    fetchSchedules()  
  } else {  
    setSchedules([])  
  }  
}, [loggedInUser, route])
```

FLATLISTs

En data poner el useState generado arriba.

VIEWS

Son cajitas que engloban muchos elementos. Se toma como un único elemento.

Dentro de ella se realizan los diseños para el alineamiento (flexDirection y justifyContent)

MODAL

Ejemplo en RestaurantScreen

Si aparece el modal directamente al meterte, el problema está en el useState del ToBeDeleted, que debe ser null en vez de una lista/objeto vacío.

Si se elimina directamente, sin aparecer, es porque en la acción del botón de delete no se ha puesto el SetToBeDeleted.

En la función de eliminar, debe aparecer el fetch correspondiente. Esto lo hacemos para que se actualice la base de datos sin el objeto eliminado.

DISEÑO

- flexDirection: row (para ponerlos en fila), column (para ponerlos en columna)
- justifyContent: flex-start (pegados al inicio), center (pegados en el centro), flex-end (pegados al final), space-between (mucho espacio, alinea solo), space-around (menos espacio, alinea solo), space-evenly (poco espacio, alinea solo)
- alignItem: flex-start (arriba de la página), center (centro de la página), flex-end (al final de la página), stretch (que ocupe toda la página).

Para los iconos se utiliza un MaterialCommunityIcon

FORMIKS

Todo debe estar dentro del View de los Inputs

UseEffect para formik de guardado

```
useEffect(() => {
  async function fetchOrder() {
    try {
      const fetchedOrder = await get (route.params.id)
      setOrder(fetchedOrder)
      setInitialOrderValues({
        address: fetchedOrder.address,
        price: fetchedOrder.price.toString()
      })
    } catch (error) {
      showMessage({
        message: `There was an error while retrieving order details. ${error}`,
        type: 'error',
        style: GlobalStyles.flashStyle,
        titleStyle: GlobalStyles.flashTextStyle
      })
    }
  }
  fetchOrder()
}, [route])
```

UseState

1. Cuándo usar [] (Un Arreglo / Array)

Úsalo **siempre que vayas a guardar una LISTA de cosas** (varios elementos).

- **Pistas de que necesitas []:**
 - El nombre de tu variable está en **plural**: orders, categories, users, addresses.
 - Vas a usar un **<FlatList>** en tu pantalla para dibujarlos. (¡El FlatList se alimenta única y exclusivamente de arreglos!).
 - Vas a usar la función `.map()` en tu código.

2. Cuándo usar {} (Un Objeto)

Úsalo **cuando vas a guardar UNA SOLA COSA**, pero esa cosa tiene varias características o atributos por dentro.

- **Pistas de que necesitas {}:**
 - El nombre de tu variable está en **singular**: restaurant, user, orderDetail, product.
 - En tu código vas a acceder a sus datos usando un punto, por ejemplo: restaurant.name, user.email, product.price.

Opción 1: Si es true -> ejecute, si es false -> no ejecute.

```
{ condición && (lo que se ejecuta si se cumple)}
```

Opción 2: Si es true -> ejecuta lo primero, si es false -> ejecuta lo segundo

```
{ condición ? (lo que se ejecuta si es true) : (lo que se ejecuta si es false)}
```

HANDLE (Estructura)

```
const handleNextStatus = async (order) => {  
  /// SOLUTION. Excercise - Order status change  
  try {  
    await nextStatus(order)  
    showMessage({  
      message: ` Order ${order.id} status updated ` ,  
      type: 'success',  
      style: GlobalStyles.flashStyle,  
      titleStyle: GlobalStyles.flashTextStyle  
    })  
    fetchRestaurantOrders()  
    fetchRestaurantAnalytics()  
  } catch (error) {  
    showMessage({  
      message: ` Error advancing order status. ${error} ` ,  
      type: 'danger',  
      style: GlobalStyles.flashStyle,  
      titleStyle: GlobalStyles.flashTextStyle  
    })  
  }  
}
```